EC200U&EG91xU Series QuecOpen(SDK) Cellular Network Information API Reference Manual

LTE Standard Module Series

Version: 1.1

Date: 2023-09-28

Status: Released

At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.
Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China
Tel: +86 21 5108 6236
Email: info@quectel.com

Or our local offices. For more information, please visit:
http://www.quectel.com/support/sales.htm.

For technical support, or to report documentation errors, please visit:
http://www.quectel.com/support/technical.htm.
Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2021-11-05	Braden HE	Creation of the document
1.0	2021-11-25	Braden HE	First official release
1.1	2023-09-28	Joe TU	1. Updated the document name based on the unified QuecOpen naming. 2. Added the applicable modules EG912U-GL and EG915U series. 3. Deleted the note of (U)SIM interface supported by different modules and added the note about the support of WCDMA and GSM networks of different modules (Chapter 1.1).

Contents

Table Index

1 Introduction

Quectel EC200U series and EG91xU family modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS, which is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see ***document [1]***.

This document is applicable to QuecOpen® solution based on CSDK build environment. It introduces the network information API, relevant enumerations and structures of EC200U series and EG91xU family modules.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC200U Series
EG91xU	EG912U-GL
	EG915U Series

NOTE

1. The EC200U series and EG91xU family modules do not support WCDMA network.
2. The GSM network is supported by EG91xU family modules, and it is optional for EC200U series module.

2 Network Information API

2.1. Header File

ql_api_nw.h, the header file of network information API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, all the header files mentioned in this document are in this directory.

2.2. Example

The example of network information API is in the *\components\ql-application\nw\nw_demo.c* directory of the CSDK.

2.3. API Overview

Table 2: API Overview

Function	Description
ql_nw_set_mode()	Sets network mode.
ql_nw_get_mode()	Gets the current network mode.
ql_nw_get_nitz_time_info()	Gets the current network time.
ql_nw_get_operator_name()	Gets the current operator information.
ql_nw_get_reg_status()	Gets network registration status.
ql_nw_get_csq()	Gets network signal quality information.
ql_nw_get_signal_strength()	Gets network signal strength information.
ql_nw_get_cell_info()	Gets information of the primary cell and neighboring cell.

ql_nw_register_cb()	Registers the network registration callback function.
ql_nw_set_selection()	Selects the operator.
ql_nw_get_selection()	Gets the current selected operator information.
ql_nw_get_data_count()	Queries the traffic consumed by uplink and downlink.
ql_nw_reset_data_count()	Clears the counted traffic.

2.4. API Description

2.4.1. ql_nw_set_mode

This function sets network mode.

- **Prototype**

```
ql_nw_errcode_e ql_nw_set_mode(uint8_t nSim, ql_nw_mode_type_e nw_mode)
```

- **Parameter**

nSim:
[In] The (U)SIM card in use. This parameter can be set to 0 if the module only supports one (U)SIM interface.
0 (U)SIM 1
1 (U)SIM 2

nw_mode:
[In] The network mode to be set. See ***Chapter 0*** for details.

- **Return Value**

See ***Chapter 2.4.1.2*** for details.

2.4.1.1. ql_nw_mode_type_e

The enumeration of network mode:

```
typedef enum
{
    QL_NW_MODE_AUTO = 0,
    QL_NW_MODE_GSM,
    QL_NW_MODE_LTE,
}ql_nw_mode_type_e
```

- **Member**

Member	Description
QL_NW_MODE_AUTO	The modem automatically selects the network mode with LTE mode preferred.
QL_NW_MODE_GSM	GSM mode.
QL_NW_MODE_LTE	LTE mode.

2.4.1.2. ql_nw_errcode_e

The enumeration of error codes of network information API:

```
typedef enum
{
    QL_NW_SUCCESS                   = 0,
    QL_NW_EXECUTE_ERR               = 1 | (QL_COMPONENT_NETWORK << 16),
    QL_NW_MEM_ADDR_NULL_ERR,
    QL_NW_INVALID_PARAM_ERR,
    QL_NW_CFW_CFUN_GET_ERR,
    QL_NW_CFUN_DISABLE_ERR          = 5 | (QL_COMPONENT_NETWORK << 16),
    QL_NW_CFW_NW_STATUS_GET_ERR,
    QL_NW_NOT_SEARCHING_ERR,
    QL_NW_NOT_REGISTERED_ERR,
    QL_NW_CFW_GPRS_STATUS_GET_ERR,
    QL_GPRS_NOT_SEARCHING_ERR   = 10 | (QL_COMPONENT_NETWORK << 16),
    QL_GPRS_NOT_REGISTERED_ERR,
    QL_NW_CFW_NW_QUAL_GET_ERR,
    QL_NW_CFW_OPER_ID_GET_ERR,
    QL_NW_CFW_OPER_NAME_GET_ERR,
    QL_NW_CFW_OPER_SET_ERR          = 15 | (QL_COMPONENT_NETWORK << 16),
    QL_NW_SIM_ERR,
```

```
    QL_NW_NO_MEM_ERR,
    QL_NW_SEMAPHORE_CREATE_ERR,
    QL_NW_SEMAPHORE_TIMEOUT_ERR,
    QL_NW_NITZ_NOT_UPDATE_ERR      = 20 | (QL_COMPONENT_NETWORK << 16),
    QL_NW_CFW_EMOD_START_ERR,
}ql_nw_errcode_e
```

- **Member**

Member	Description
QL_NW_SUCCESS	Successful execution.
QL_NW_EXECUTE_ERR	Failed execution.
QL_NW_MEM_ADDR_NULL_ERR	The API parameter address is null.
QL_NW_INVALID_PARAM_ERR	Invalid API parameter.
QL_NW_CFW_CFUN_GET_ERR	Failed to get function mode.
QL_NW_CFUN_DISABLE_ERR	Non-full function mode, resulting in related information unavailable.
QL_NW_CFW_NW_STATUS_GET_ERR	Failed to get network status, resulting in related information unavailable.
QL_NW_NOT_SEARCHING_ERR	The operator to be registered is not found, resulting in related information unavailable.
QL_NW_NOT_REGISTERED_ERR	The network is not registered, resulting in related information unavailable.
QL_NW_CFW_GPRS_STATUS_GET_ERR	Failed to get PS domain network status.
QL_GPRS_NOT_SEARCHING_ERR	No operator to be registered for PS domain is found.
QL_GPRS_NOT_REGISTERED_ERR	Not registered on a PS domain network.
QL_NW_CFW_NW_QUAL_GET_ERR	Failed to get signal quality.
QL_NW_CFW_OPER_ID_GET_ERR	Failed to get operator ID.
QL_NW_CFW_OPER_NAME_GET_ERR	Failed to get operator name.
QL_NW_CFW_OPER_SET_ERR	Failed to set operator.
QL_NW_SIM_ERR	(U)SIM card related error, resulting in information unavailable.
QL_NW_NO_MEM_ERR	Failed to allocate memory.
QL_NW_SEMAPHORE_CREATE_ERR	Failed to create semaphore.

QL_NW_SEMAPHORE_TIMEOUT_ERR	Waiting for semaphore timeout.
QL_NW_NITZ_NOT_UPDATE_ERR	Unsynchronized network time.
QL_NW_CFW_EMOD_START_ERR	Failed to enter engineering mode, resulting in cell information unavailable.

2.4.2. ql_nw_get_mode

This function gets the current network mode.

- **Prototype**

```
ql_nw_errcode_e ql_nw_get_mode(uint8_t nSim, ql_nw_mode_type_e *nw_mode)
```

- **Parameter**

nSim:
[In] The (U)SIM card in use. This parameter can be set to 0 if the module only supports one (U)SIM interface.
0 (U)SIM 1
1 (U)SIM 2

nw_mode:
[Out] The current network mode. See ***Chapter 0*** for details.

- **Return Value**

See ***Chapter 2.4.1.2*** for details.

2.4.3. ql_nw_get_nitz_time_info

This function gets the current network time.

- **Prototype**

```
ql_nw_errcode_e ql_nw_get_nitz_time_info(ql_nw_nitz_time_info_s *nitz_info)
```

- **Parameter**

nitz_info:
[Out] The current network time. See ***Chapter 2.4.3.1*** for details.

- **Return Value**

See ***Chapter 2.4.1.2*** for details.

2.4.3.1. ql_nw_nitz_time_info_s

The structure of the current network time:

```
typedef struct
{
   char         nitz_time[32];
   long         abs_time;
}ql_nw_nitz_time_info_s;
```

- **Parameter**

Type	Parameter	Description
char	*nitz_time*	Local time. Format: YY/MM/DD HH:MM:SS '+/-'TZ DST TZ: local zone (displaying the difference between local time and GMT time in unit of 15 minutes) DST: Daylight Saving Time. For example: 20/10/21 09:17:43 +32 00
long	*abs_time*	Absolute time, relative to 00:00:00 on January 1, 1970 (UTC). *0* indicates unavailability.

2.4.4. ql_nw_get_operator_name

This function gets the current operator information.

- **Prototype**

```
ql_nw_errcode_e ql_nw_get_operator_name(uint8_t nSim, ql_nw_operator_info_s *oper_info)
```

- **Parameter**

nSim:
[In] The (U)SIM card in use. This parameter can be set to 0 if the module only supports one (U)SIM interface.
0 (U)SIM 1
1 (U)SIM 2

oper_info:
[Out] The operator information. See ***Chapter 2.4.4.1*** for details.

- **Return Value**

See ***Chapter 2.4.1.2*** for details.

2.4.4.1. ql_nw_operator_info_s

The structure of operator information:

```
typedef struct
{
    char long_oper_name[QL_NW_LONG_OPER_MAX_LEN+1];
    char short_oper_name[QL_NW_SHORT_OPER_MAX_LEN+1];
    char mcc[QL_NW_MCC_MAX_LEN+1];
    char mnc[QL_NW_MNC_MAX_LEN+1];
}ql_nw_operator_info_s
```

- **Parameter**

Type	Parameter	Description
char	*long_oper_name*	Full name of operator.
char	*short_oper_name*	Short name of operator.
char	*mcc*	Mobile country code.
char	*mnc*	Mobile network code.

2.4.5. ql_nw_get_reg_status

This function gets network registration status, including that of voice call and data call.

- **Prototype**

```
ql_nw_errcode_e ql_nw_get_reg_status(uint8_t nSim, ql_nw_reg_status_info_s *reg_info)
```

- **Parameter**

nSim:
[In] The (U)SIM card in use. This parameter can be set to 0 if the module only supports one (U)SIM interface.
0 (U)SIM 1
1 (U)SIM 2

reg_info:
[Out] Network registration status. See ***Chapter 2.4.5.1*** for details.

- **Return Value**

See ***Chapter 2.4.1.2*** for details.

2.4.5.1. ql_nw_reg_status_info_s

The structure of network registration status:

```
typedef struct
{
    ql_nw_common_reg_status_info_s    voice_reg;
    ql_nw_common_reg_status_info_s    data_reg;
}ql_nw_reg_status_info_s
```

- **Parameter**

Type	Parameter	Description
ql_nw_common_reg_status_info_s	*voice_reg*	Network registration status of voice call. See ***Chapter 2.4.5.2*** for details.
ql_nw_common_reg_status_info_s	*data_reg*	Network registration status of data call. See ***Chapter 2.4.5.2*** for details.

2.4.5.2. ql_nw_common_reg_status_info_s

The structure of network registration status of voice call or data call:

```
typedef struct
{
    ql_nw_reg_state_e    state;
    int                  lac;
    int                  cid;
    ql_nw_act_type_e     act;
}ql_nw_common_reg_status_info_s
```

- **Parameter**

Type	Parameter	Description
ql_nw_reg_state_e	*state*	Network registration status. See ***Chapter 2.4.5.3*** for details.
int	*lac*	Location area code.
int	*cid*	Cell ID.
ql_nw_act_type_e	*act*	Network registration mode. See ***Chapter 2.4.5.4*** for details.

2.4.5.3. ql_nw_reg_state_e

The enumeration of network registration status:

```
typedef enum
{
  QL_NW_REG_STATE_NOT_REGISTERED                =0,
  QL_NW_REG_STATE_HOME_NETWORK                  =1,
  QL_NW_REG_STATE_TRYING_ATTACH_OR_SEARCHING    =2,
  QL_NW_REG_STATE_DENIED                        =3,
  QL_NW_REG_STATE_UNKNOWN                       =4,
  QL_NW_REG_STATE_ROAMING                       =5,
}ql_nw_reg_state_e
```

- **Member**

Member	Description
QL_NW_REG_STATE_NOT_REGISTERED	Not registered. MT is not currently searching an operator to register on network.
QL_NW_REG_STATE_HOME_NETWORK	Registered: home network.
QL_NW_REG_STATE_TRYING_ATTACH_OR_SEARCHING	Not registered, but MT is currently trying to register on network or searching an operator to register on network.
QL_NW_REG_STATE_DENIED	Registration denied.
QL_NW_REG_STATE_UNKNOWN	Unknown.
QL_NW_REG_STATE_ROAMING	Registered: roaming.

2.4.5.4. ql_nw_act_type_e

The enumeration of network registration mode:

```
typedef enum
{
    QL_NW_ACCESS_TECH_GSM                 = 0,
    QL_NW_ACCESS_TECH_GSM_COMPACT         = 1,
    QL_NW_ACCESS_TECH_UTRAN               = 2,
    QL_NW_ACCESS_TECH_GSM_wEGPRS          = 3,
    QL_NW_ACCESS_TECH_UTRAN_wHSDPA        = 4,
    QL_NW_ACCESS_TECH_UTRAN_wHSUPA        = 5,
    QL_NW_ACCESS_TECH_UTRAN_wHSDPA_HSUPA  = 6,
```

```
    QL_NW_ACCESS_TECH_E_UTRAN                    = 7,
}ql_nw_act_type_e;
```

- **Member**

Member	Description
QL_NW_ACCESS_TECH_GSM	GSM (2G).
QL_NW_ACCESS_TECH_GSM_COMPACT	GSM COMPACT (2G): unsupported.
QL_NW_ACCESS_TECH_UTRAN	UTRAN (3G): unsupported.
QL_NW_ACCESS_TECH_GSM_wEGPRS	EGPRS (2G): unsupported.
QL_NW_ACCESS_TECH_UTRAN_wHSDPA	HSDPA (3G): unsupported.
QL_NW_ACCESS_TECH_UTRAN_wHSUPA	HSUPA (3G): unsupported.
QL_NW_ACCESS_TECH_UTRAN_wHSDPA_HSUPA	HSDPA (3G) and HSUPA (3G): unsupported.
QL_NW_ACCESS_TECH_E_UTRAN	E-UTRAN (4G).

2.4.6. ql_nw_get_csq

This function gets network signal quality information.

- **Prototype**

```
ql_nw_errcode_e ql_nw_get_csq(uint8_t nSim, unsigned char *csq)
```

- **Parameter**

nSim:
[In] The (U)SIM card in use. This parameter can be set to 0 if the module only supports one (U)SIM interface.
0 (U)SIM 1
1 (U)SIM 2

csq:
[Out] The returned network signal quality information. Range: 0–31. 99 indicates invalid value.

- **Return Value**

See ***Chapter 2.4.1.2*** for details.

2.4.7. ql_nw_get_signal_strength

This function gets network signal strength information.

- **Prototype**

```
ql_nw_errcode_e ql_nw_get_signal_strength(uint8_t nSim, ql_nw_signal_strength_info_s *pt_info)
```

- **Parameter**

nSim:
[In] The (U)SIM card in use. This parameter can be set to 0 if the module only supports one (U)SIM interface.
0 (U)SIM 1
1 (U)SIM 2

pt_info:
[Out] Network signal strength information. See ***Chapter 2.4.7.1*** for details.

- **Return Value**

See ***Chapter 2.4.1.2*** for details.

2.4.7.1. ql_nw_signal_strength_info_s

The structure of network signal strength information:

```
typedef struct
{
    int rssi;
    int bitErrorRate;
    int rsrq;
    int rsrp;
}ql_nw_signal_strength_info_s
```

- **Parameter**

Type	Parameter	Description
int	*rssi*	Received signal strength indicator. 99 indicates invalid value.
int	*bitErrorRate*	The channel bit error rate (BER), valid only during a call. 99 indicates invalid value.
int	*rsrq*	Reference signal receiving quality, used in LTE mode. 255 indicates invalid value.

int	*rsrp*	Reference signal receiving power, used in LTE mode. 255 indicates invalid value.

2.4.8. ql_nw_get_cell_info

This function gets information of the primary cell and neighboring cell.

- **Prototype**

```
ql_nw_errcode_e ql_nw_get_cell_info(uint8_t nSim, ql_nw_cell_info_s *cell_info)
```

- **Parameter**

nSim:
[In] The (U)SIM card in use. This parameter can be set to 0 if the module only supports one (U)SIM interface.
0 (U)SIM 1
1 (U)SIM 2

cell_info:
[Out] Information of the primary cell and neighboring cell. See ***Chapter 2.4.8.1*** for details.

- **Return Value**

See ***Chapter 2.4.1.2*** for details.

2.4.8.1. ql_nw_cell_info_s

The structure of cell information:

```
typedef  struct
{
    int                         gsm_info_valid;
    int                         gsm_info_num;
    ql_nw_gsm_cell_info_s   gsm_info[QL_NW_CELL_MAX_NUM];
    int                         lte_info_valid;
    int                         lte_info_num;
    ql_nw_lte_cell_info_s       lte_info[QL_NW_CELL_MAX_NUM];
}ql_nw_cell_info_s
```

- **Parameter**

Type	Parameter	Description
int	*gsm_info_valid*	Whether cell information is available in GSM mode. *0* Unavailable *1* Available
int	*gsm_info_num*	The number of cells gotten in GSM mode.
ql_nw_gsm_cell_info_s	*gsm_info*	List of cell information gotten in GSM mode. See ***Chapter 2.4.8.2*** for details.
int	*lte_info_valid*	Whether cell information is available in LTE mode. *0* Unavailable *1* Available
int	*lte_info_num*	The number of cells gotten in LTE mode.
ql_nw_lte_cell_info_s	*lte_info*	List of cell information gotten in LTE mode. See ***Chapter 2.4.8.3*** for details.

2.4.8.2. ql_nw_gsm_cell_info_s

The structure of cell information gotten in GSM mode:

```
typedef struct
{
    int flag;
    int cid;
    int mcc;
    int mnc;
    int lac;
    int arfcn;
    char bsic;
    int rssi;
    char mnc_len;
    char RX_dBm;
}ql_nw_gsm_cell_info_s
```

- **Parameter**

Type	Parameter	Description
int	*flag*	Cell type. *0* Primary cell *1* Neighboring cell
int	*cid*	Cell ID. *0* indicates that the cell ID is not received.

int	*mcc*	Mobile country code.
int	*mnc*	Mobile network code.
int	*lac*	Location area code.
int	*arfcn*	Absolute RF channel number.
char	*bsic*	Base station identity code. *0* indicates the base station identity code is not received.
int	*rssi*	Received signal strength indicator. Unit: dBm.
char	*mnc_len*	Mobile network code length.
char	*RX_dBm*	Received power. Unit: dBm.

2.4.8.3. ql_nw_lte_cell_info_s

The structure of cell information gotten in LTE mode:

```
typedef struct
{
    int flag;
    int cid;
    int mcc;
    int mnc;
    int tac;
    int pci;
    int earfcn;
    int rssi;
    char mnc_len;
    char RX_dBm;
}ql_nw_lte_cell_info_s
```

- **Parameter**

Type	Parameter	Description
int	*flag*	Cell type. *0* Primary cell *1* Neighboring cell
int	*cid*	Cell ID. *0* indicates that the cell ID is not received.
int	*mcc*	Mobile country code.
int	*mnc*	Mobile network code.

int	*tac*	Tracking area code.
int	*pci*	Physical cell identification code.
int	*earfcn*	EARFCN. Range: 0–65535.
int	*rssi*	Received signal strength indicator. Unit: dBm.
char	*mnc_len*	Mobile network code length.
char	*RX_dBm*	Received power. Unit: dBm.

2.4.9. ql_nw_register_cb

This function registers the network registration callback function.

- **Prototype**

```
ql_nw_errcode_e ql_nw_register_cb(ql_nw_callback nw_cb)
```

- **Parameter**

nw_cb:
[In] Network registration callback function. See ***Chapter 2.4.9.1*** for details. If the parameter is NULL, it indicates that the thread cancels the callback function registration.

- **Return Value**

See ***Chapter 2.4.1.2*** for details.

2.4.9.1. ql_nw_callback

This function is network registration callback function.

- **Prototype**

```
typedef void (*ql_nw_callback)(uint8_t nSim, unsigned int ind_type, void *ctx)
```

- **Parameter**

nSim:
[In] The (U)SIM card in use. This parameter can be set to 0 if the module only supports one (U)SIM interface.
0 (U)SIM 1
1 (U)SIM 2

ind_type:
[In] Network registration event type.

QUEC_NW_SIGNAL_QUALITY_IND	Signal strength event
QUEC_NW_VOICE_REG_STATUS_IND	Voice call event
QUEC_NW_DATA_REG_STATUS_IND	Data call event
QUEC_NW_NITZ_TIME_UPDATE_IND	Network time update event

ctx:
[In] Information data corresponding to events.

- **Return Value**

None

2.4.10. ql_nw_set_selection

This function selects the operator.

- **Prototype**

```
ql_nw_errcode_e ql_nw_set_selection(uint8_t nSim, ql_nw_seclection_info_s *select_info)
```

- **Parameter**

nSim:
[In] The (U)SIM card in use. This parameter can be set to 0 if the module only supports one (U)SIM interface.

0	(U)SIM 1
1	(U)SIM 2

select_info:
[In] The selected operator information. See ***Chapter 2.4.10.1*** for details.

- **Return Value**

See ***Chapter 2.4.1.2*** for details.

2.4.10.1. ql_nw_seclection_info_s

The structure of the selected operator information:

```
typedef  struct
{
    unsigned  char       nw_selection_mode;
```

```
    char                mcc[QL_NW_MCC_MAX_LEN+1];
    char                mnc[QL_NW_MNC_MAX_LEN+1];
    ql_nw_act_type_e    act;
}ql_nw_seclection_info_s
```

- **Parameter**

Type	Parameter	Description
unsigned char	*nw_selection_mode*	Operator selection modes. *0* Automatically selected by modem. MCC and MNC are invalid. *1* Manually selected.
char	*mcc*	Mobile country code.
char	*mnc*	Mobile network code.
ql_nw_act_type_e	*act*	Network registration mode. See ***Chapter 2.4.5.4*** for details.

2.4.11. ql_nw_get_selection

This function gets the current selected operator information.

- **Prototype**

```
ql_nw_errcode_e ql_nw_get_selection(uint8_t nSim, ql_nw_seclection_info_s *select_info)
```

- **Parameter**

nSim:
[In] The (U)SIM card in use. This parameter can be set to 0 if the module only supports one (U)SIM interface.
0 (U)SIM 1
1 (U)SIM 2

select_info:
[Out] The operator information. See ***Chapter 2.4.10.1*** for details.

- **Return Value**

See ***Chapter 2.4.1.2*** for details.

2.4.12. ql_nw_get_data_count

This function queries the traffic consumed by uplink and downlink.

- **Prototype**

```
ql_nw_errcode_e ql_nw_get_data_count(uint8_t nSim, ql_nw_data_count_info_s * data_info)
```

- **Parameter**

nSim:
[In] The (U)SIM card in use. This parameter can be set to 0 if the module only supports one (U)SIM interface.
0 (U)SIM 1
1 (U)SIM 2

data_info:
[Out] The counted traffic information. See ***Chapter 2.4.12.1*** for details.

- **Return Value**

See ***Chapter 2.4.1.2*** for details.

2.4.12.1. ql_nw_data_count_info_s

The structure of counted traffic information:

```
typedef  struct
{
    uint64_t  uplink_data_count;
    uint64_t  downlink_data_count;
}ql_nw_data_count_info_s
```

- **Parameter**

Type	Parameter	Description
uint64_t	*uplink_data_count*	Traffic consumed by uplink. Unit: byte.
uint64_t	*downlink_data_count*	Traffic consumed by downlink. Unit: byte.

2.4.13. ql_nw_reset_data_count

This function clears the counted traffic.

- **Prototype**

```
ql_nw_errcode_e ql_nw_reset_data_count(uint8_t nSim)
```

- **Parameter**

nSim:

[In] The (U)SIM card in use. This parameter can be set to 0 if the module only supports one (U)SIM interface.

0 (U)SIM 1

1 (U)SIM 2

- **Return Value**

See ***Chapter 2.4.1.2*** for details.

3 Appendix References

Table 3: Related Document

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Quick_Start_Guide

Table 4: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
ARFCN	Absolute Radio Frequency Channel Number
BSIC	Base Station Identity Code
EARFCN	E-UTRA Absolute Radio Frequency Channel Number
EGPRS	Enhanced General Packet Radio Service
E-UTRAN	Evolved Universal Terrestrial Radio Access Network
GSM	Global System for Mobile Communications
HSDPA	High Speed Downlink Packet Access
HSUPA	High Speed Uplink Packet Access
ID	Identifier
IoT	Internet of Things
LAC	Location Area Code
LTE	Long-Term Evolution
MCC	Mobile Country Code

MNC	Mobile Network Code
MT	Mobile Termination
NITZ	Network Identity and Time Zone
PCI	Physical Cell Identifier
PS	Packet Switch
RSRP	Reference Signal Receiving Power
RSRQ	Reference Signal Receiving Quality
RSSI	Received Signal Strength Indicator
RTOS	Real-Time Operating System
RX	Receive
(U)SIM	(Universal) Subscriber Identity Module
TAC	Tracking Area Code
UTRAN	Universal Terrestrial Radio Access Network